AWS

VPC Peering

Connect two VPCs. Prove private connectivity.

A bastion host, custom route tables, security groups and a VPC peering connection in Asia Pacific (Sydney).

Field	Details
Name	Mohamed Afsar
Batch	AWS CLOUD INTERNSHIP
Project title	AWS VPC Peering (Project 01)
Region	Asia Pacific (Sydney), ap-southeast-2
Completion date	30 October 2026
Report file	Mohamed_Afsar_EWB_VPC_Peering.pdf

1. Project overview

EWB has an application in VPC A and a second workload in VPC B. Both application servers must stay private. The goal of this project was to connect the two VPCs with a VPC peering connection so that the servers can exchange traffic over their private IPv4 addresses, with one public bastion host in VPC A used for administration.

What was built

- Two VPCs with non-overlapping CIDR ranges: ewb-vpc-a (10.10.0.0/16) and ewb-vpc-b (10.20.0.0/16).
- Three subnets with explicit route table associations: ewb-a-public, ewb-a-private and ewb-b-private.
- One internet gateway (ewb-igw-a), attached to VPC A only. VPC B has no internet gateway.
- Three custom route tables: ewb-rt-a-public, ewb-rt-a-private and ewb-rt-b-private.
- Three security groups scoped to individual lab hosts: ewb-sg-bastion, ewb-sg-app-a and ewb-sg-app-b.
- Three EC2 instances (Amazon Linux 2023, t3.micro): ewb-bastion (public), ewb-app-a and ewb-app-b (private, no public IP).
- One VPC peering connection, ewb-a-b-peer, with forward and return routes.

Results at a glance

Test	Result	Evidence
01 Baseline	App-A to App-B ping: 4 sent, 0 received, 100% loss before the peering routes existed	Figure 21
02 Forward path	App-A to App-B ping: 4 received, 0% loss	Figure 22
03 Reverse path	App-B to App-A ping: 4 received, 0% loss	Figure 23
04 Private SSH	Shell on App-B reached through the bastion (ProxyJump)	Figure 23
05 Isolation	No public IPv4 address on App-A or App-B	Figure 16
Controlled failure	Echo Request rule removed: 100% loss. Rule restored: 0% loss	Figures 24 to 26

2. Architecture and resource inventory

Administration goes from the laptop to the bastion and then to App-A or App-B using SSH ProxyJump. Project traffic between App-A and App-B uses the private IP addresses and the peering connection.

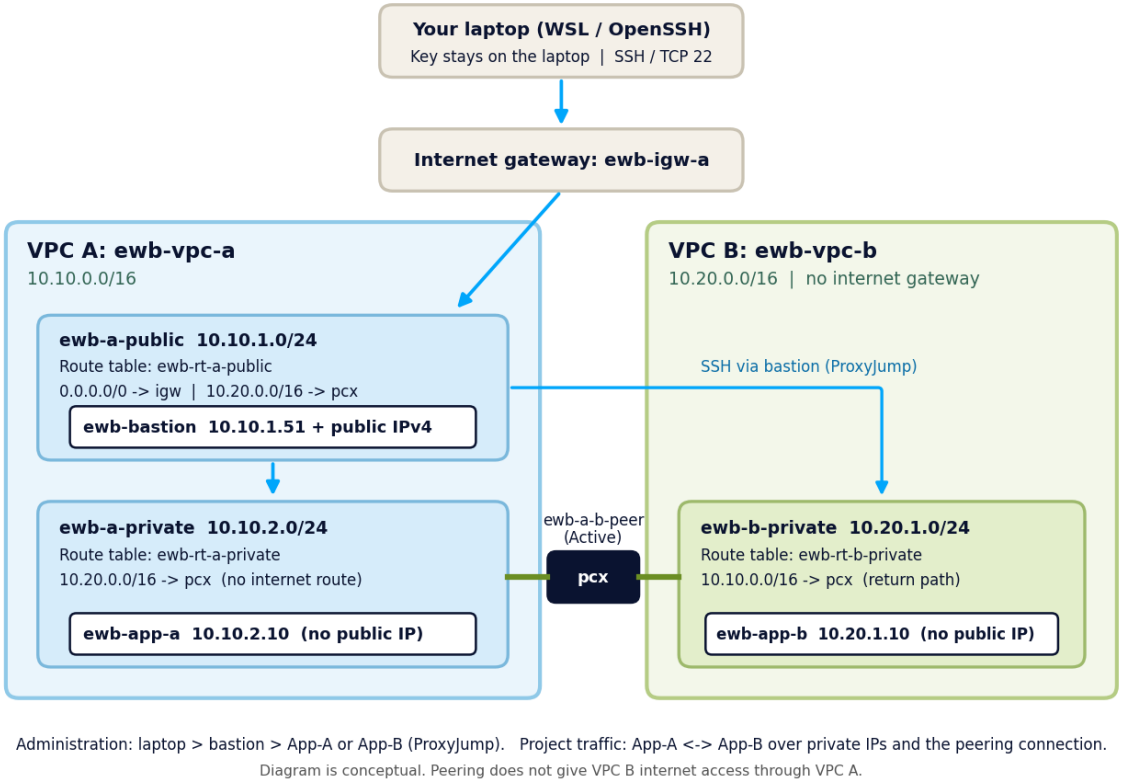


Figure 1. *Architecture of the network: public bastion in VPC A, private application servers in both VPCs, joined by the peering connection.*

Address plan

Resource	VPC / subnet	IPv4 CIDR / address
ewb-vpc-a	VPC A	10.10.0.0/16
ewb-a-public	VPC A, public subnet	10.10.1.0/24
ewb-a-private	VPC A, private subnet	10.10.2.0/24
ewb-vpc-b	VPC B	10.20.0.0/16
ewb-b-private	VPC B, private subnet	10.20.1.0/24
ewb-bastion	ewb-a-public	10.10.1.51 + public IPv4
ewb-app-a	ewb-a-private	10.10.2.10, no public IP
ewb-app-b	ewb-b-private	10.20.1.10, no public IP

The bastion received the AWS-assigned private address 10.10.1.51 instead of 10.10.1.10. As the workbook allows, every security rule that refers to the bastion uses its actual address (10.10.1.51/32).

Resource IDs

Resource	ID	Notes
ewb-vpc-a	`vpc-08e46c6fc44a80e10`	10.10.0.0/16
ewb-vpc-b	`vpc-0bcb93fdf367dd57e`	10.20.0.0/16
ewb-a-public	`subnet-01091b67dcd077b7b`	ap-southeast-2a
ewb-a-private	`subnet-0ccf2bab90dc15445`	ap-southeast-2a
ewb-b-private	`subnet-0a3b01cc7413124ad`	ap-southeast-2a
ewb-igw-a	`igw-0500b23fabd0d8ceb`	Attached to ewb-vpc-a
ewb-rt-a-public	`rtb-06b0f0bae16f32d65`	Associated: ewb-a-public
ewb-rt-a-private	`rtb-03ff9e2a4c059af5a`	Associated: ewb-a-private
ewb-rt-b-private	`rtb-0facf80417d9da653`	Associated: ewb-b-private
ewb-sg-bastion	`sg-03fbe71077976b502`	ewb-vpc-a
ewb-sg-app-a	`sg-0a2b1ca8a9dac8459`	ewb-vpc-a
ewb-sg-app-b	`sg-05bfedf496d72f1e1`	ewb-vpc-b
ewb-bastion	`i-04e40ac21d6741aa7`	10.10.1.51, public IPv4
ewb-app-a	`i-057df6a293b682b26`	10.10.2.10
ewb-app-b	`i-04cde3db61eff7aa8`	10.20.1.10
ewb-a-b-peer	`pcx-027c75511f81fcddb`	Requester VPC A, accepter VPC B

All resources carry the tags Project = EWB-VPC-Peering and Student = the student name. The account ID and public IP addresses are blurred in the screenshots.

3. Network build

3.1 VPC A and its subnets

ewb-vpc-a was created with the CIDR 10.10.0.0/16 and no IPv6. The resource map shows both subnets, ewb-a-public and ewb-a-private, in ap-southeast-2a. The VPC list shows the two lab VPCs next to the default VPC, which was left untouched.

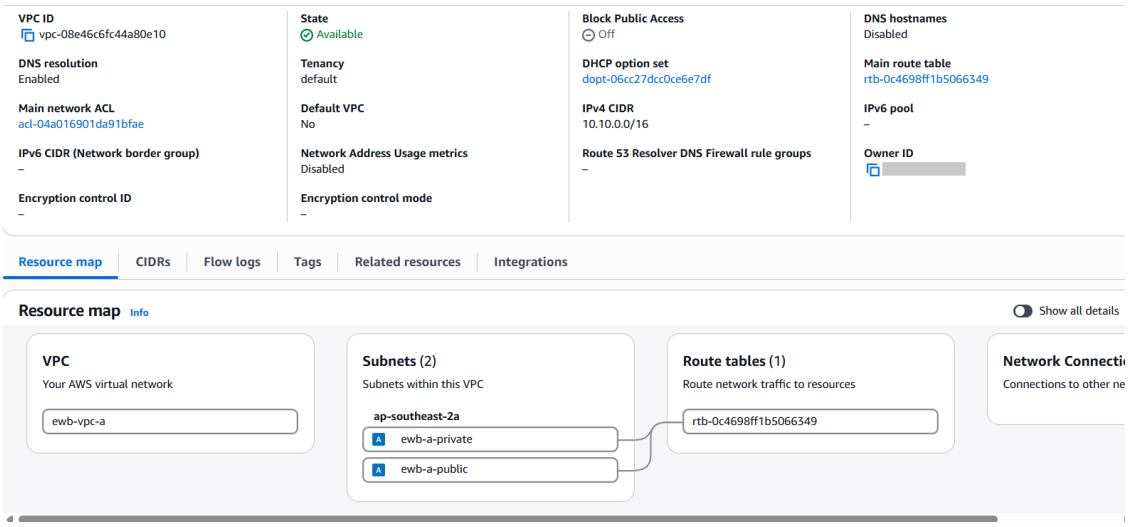


Figure 2. *ewb-vpc-a: VPC ID, state, CIDR 10.10.0.0/16 and its two subnets (resource map).*

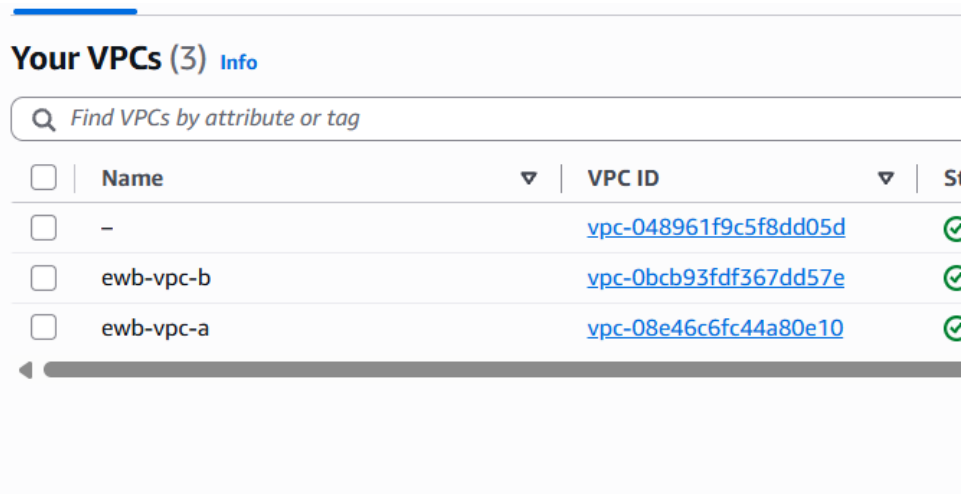


Figure 3. *Your VPCs: ewb-vpc-a and ewb-vpc-b beside the default VPC (the unnamed row).*

3.2 VPC B and subnet list

ewb-vpc-b was created with 10.20.0.0/16. It contains one private subnet, ewb-b-private (10.20.1.0/24), and has no internet gateway.

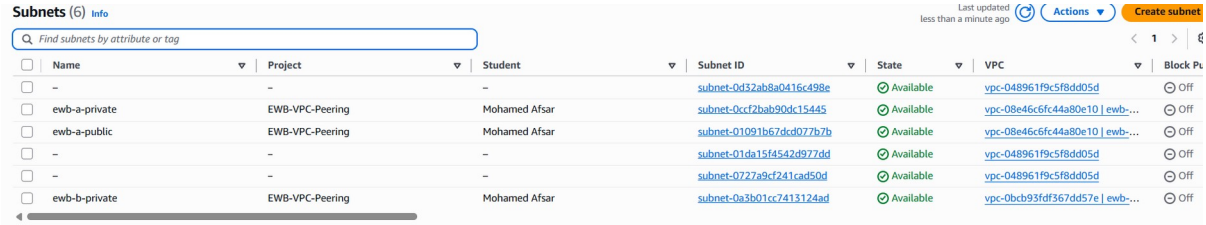


Figure 4. *Subnets list: ewb-a-private, ewb-a-public and ewb-b-private carry the project tags.*

3.3 Internet gateway

ewb-igw-a was created and attached to ewb-vpc-a. An internet gateway belongs to the VPC and is used only as a route target. The bastion is the only instance that uses it.

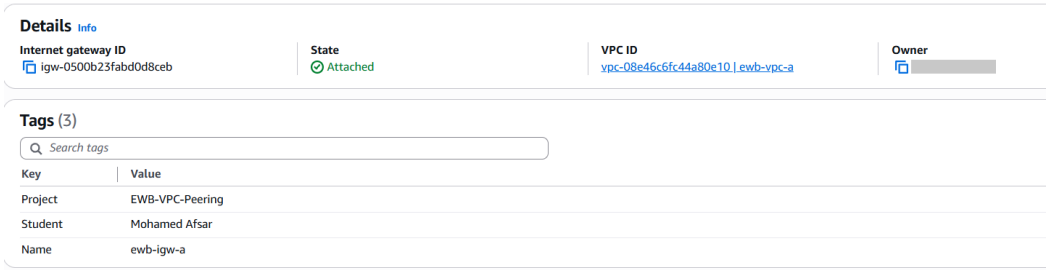


Figure 5. *ewb-igw-a: state Attached to ewb-vpc-a, with the project tags.*

3.4 Route tables and subnet associations

Each subnet is associated explicitly with its own route table, so no subnet depends on the main route table.

Route table	Associated subnet	Routes
ewb-rt-a-public	ewb-a-public	10.10.0.0/16 local 0.0.0.0/0 to ewb-igw-a 10.20.0.0/16 to the peering connection
ewb-rt-a-private	ewb-a-private	10.10.0.0/16 local 10.20.0.0/16 to the peering connection
ewb-rt-b-private	ewb-b-private	10.20.0.0/16 local 10.10.0.0/16 to the peering connection

Checkpoint 01: the private route tables were created with only their local route and each private subnet was associated explicitly.

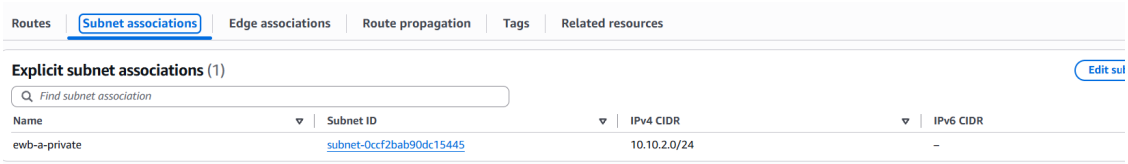


Figure 6. *ewb-rt-a-private: explicit association with ewb-a-private (10.10.2.0/24).*

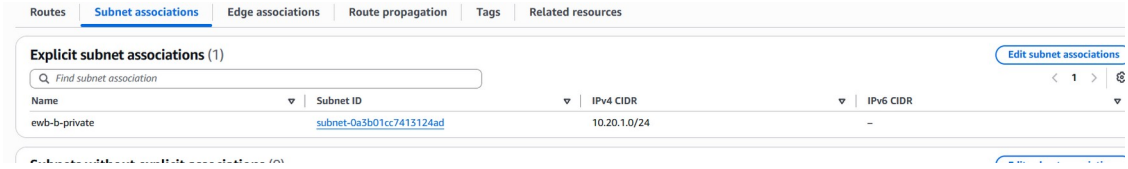


Figure 7. *ewb-rt-b-private: explicit association with ewb-b-private (10.20.1.0/24).*

Checkpoint 04: the final routes on all three tables, after the peering connection was active.

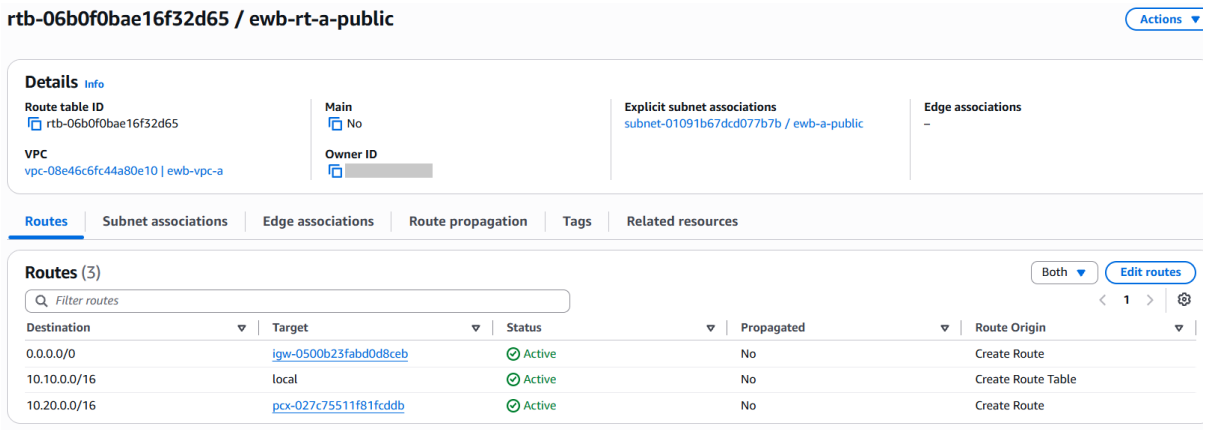


Figure 8. *ewb-rt-a-public: explicit association with ewb-a-public; routes local, 0.0.0.0/0 to the internet gateway and 10.20.0.0/16 to the peering connection (bastion path to App-B).*

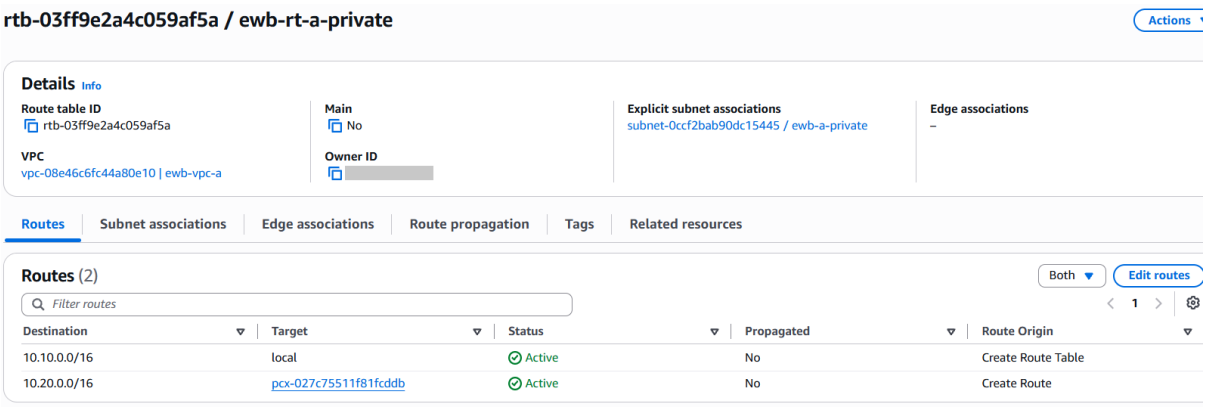


Figure 9. *ewb-rt-a-private: associated with ewb-a-private; routes local and 10.20.0.0/16 to the peering connection (App-A to VPC B).*

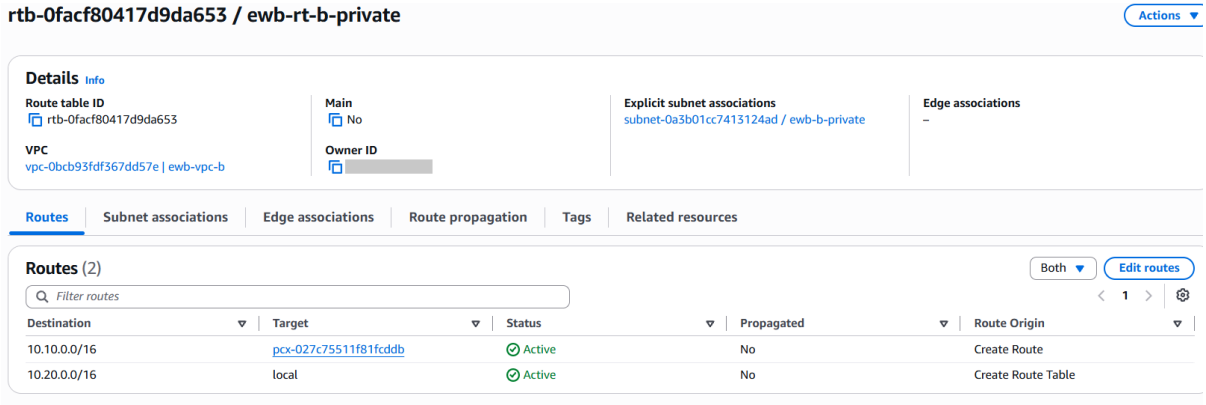


Figure 10. *ewb-rt-b-private: associated with ewb-b-private; routes 10.10.0.0/16 to the peering connection (return path to VPC A) and local.*

Why does the public subnet need a peering route?

ProxyJump makes the bastion open its own connection to App-B, so the bastion subnet must know how to reach VPC B. The more specific 10.20.0.0/16 route is chosen instead of the 0.0.0.0/0 internet route.

4. Security groups

Inbound access is limited to the lab hosts. SSH and ICMP are never opened to 0.0.0.0/0. Outbound rules were left at the default (all traffic). Security groups are stateful, so replies to permitted traffic are allowed automatically.

Security group	Type / port	Source
ewb-sg-bastion (VPC A)	SSH, TCP 22	Student public IPv4 /32 (blurred)
ewb-sg-app-a (VPC A)	SSH, TCP 22	10.10.1.51/32 (bastion private IP)
ewb-sg-app-a (VPC A)	Custom ICMP IPv4, Echo Request	10.20.1.10/32 (App-B)
ewb-sg-app-b (VPC B)	SSH, TCP 22	10.10.1.51/32 (bastion private IP)
ewb-sg-app-b (VPC B)	Custom ICMP IPv4, Echo Request	10.20.2.10/32 (App-A)

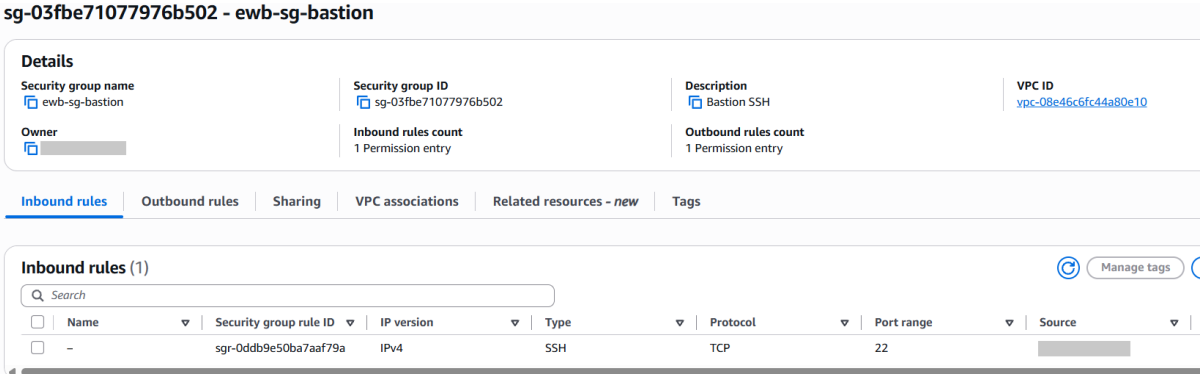


Figure 11. *ewb-sg-bastion: inbound SSH on port 22 from a single /32 address (blurred).*

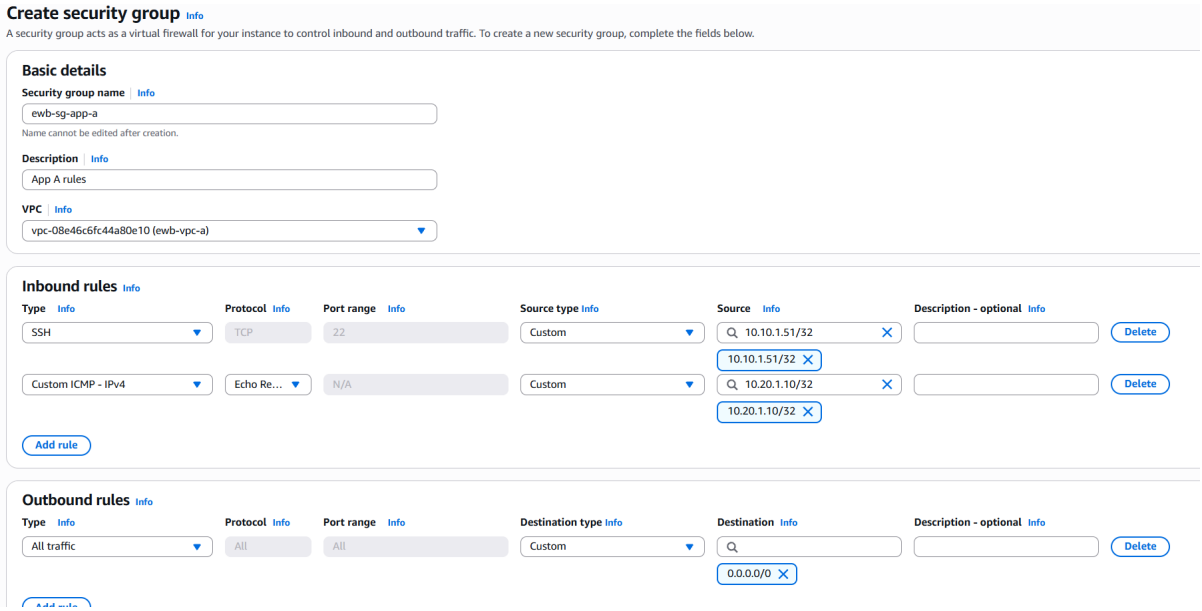


Figure 12. *ewb-sg-app-a: inbound SSH from the bastion 10.10.1.51/32 and ICMP Echo Request from App-B 10.20.1.10/32; outbound all traffic.*

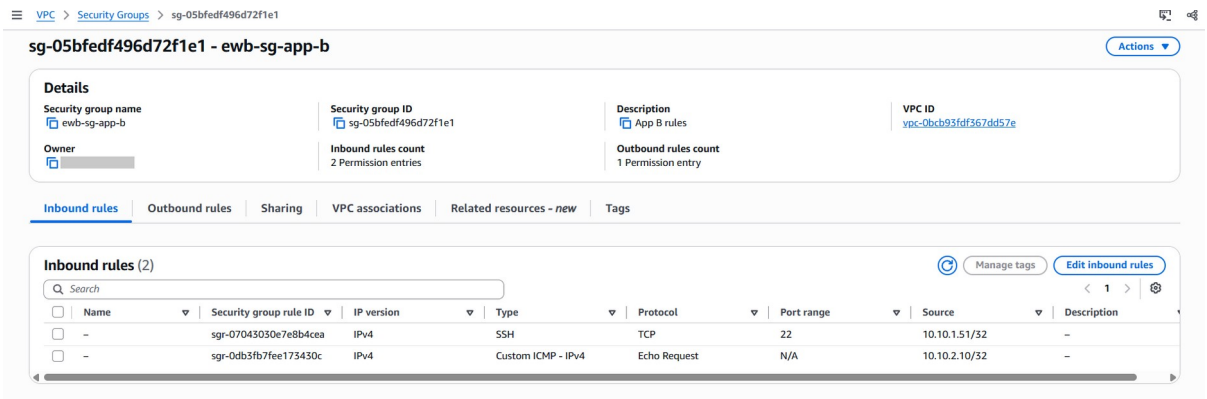


Figure 13. *ewb-sg-app-b: inbound rules SSH from the bastion 10.10.1.51/32 and ICMP Echo Request from App-A 10.10.2.10/32.*

> **Port and source are different fields**
>
> SSH uses port 22. The private IP goes in Source. App-B receives SSH from the bastion private IP because ProxyJump opens a new connection from the bastion.

5. EC2 instances

All three instances use Amazon Linux 2023 and t3.micro with the same key pair. The private key stays on the laptop only. The private servers were launched with auto-assign public IP disabled and a fixed primary private IP.

Instance	Network settings	Security group
ewb-bastion	ewb-vpc-a / ewb-a-public Private IP 10.10.1.51 Auto-assign public IP: Enable	ewb-sg-bastion
ewb-app-a	ewb-vpc-a / ewb-a-private Private IP 10.10.2.10 Auto-assign public IP: Disable	ewb-sg-app-a
ewb-app-b	ewb-vpc-b / ewb-b-private Private IP 10.20.1.10 Auto-assign public IP: Disable	ewb-sg-app-b

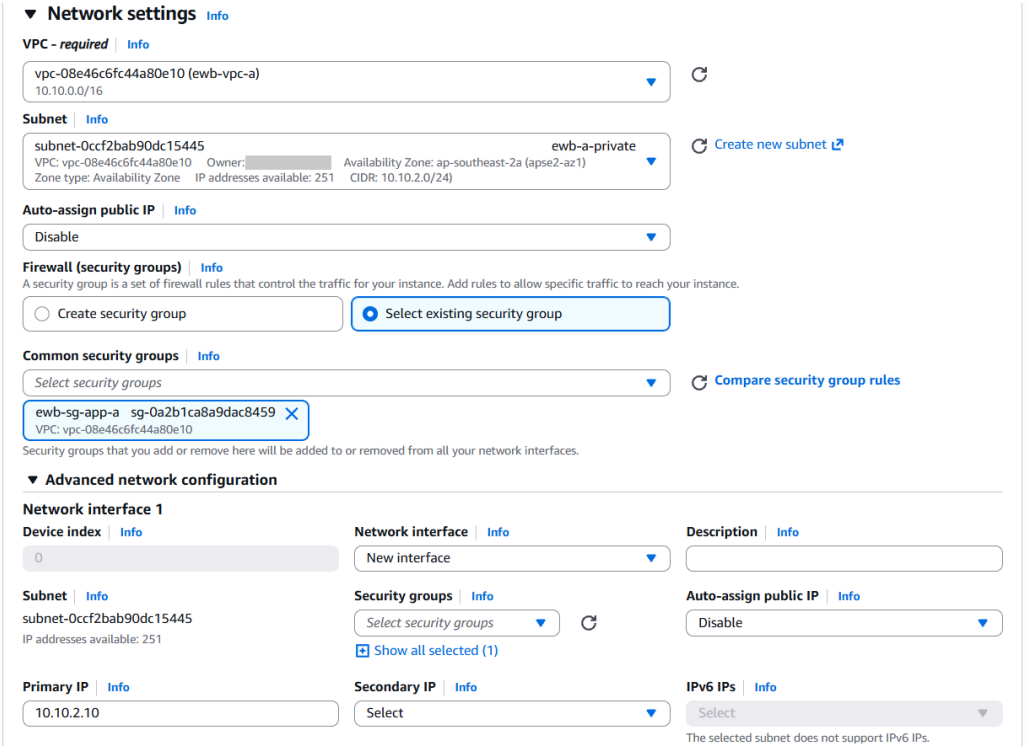


Figure 14. *ewb-app-a launch settings: ewb-vpc-a, ewb-a-private, no public IP, ewb-sg-app-a, primary IP 10.10.2.10.*

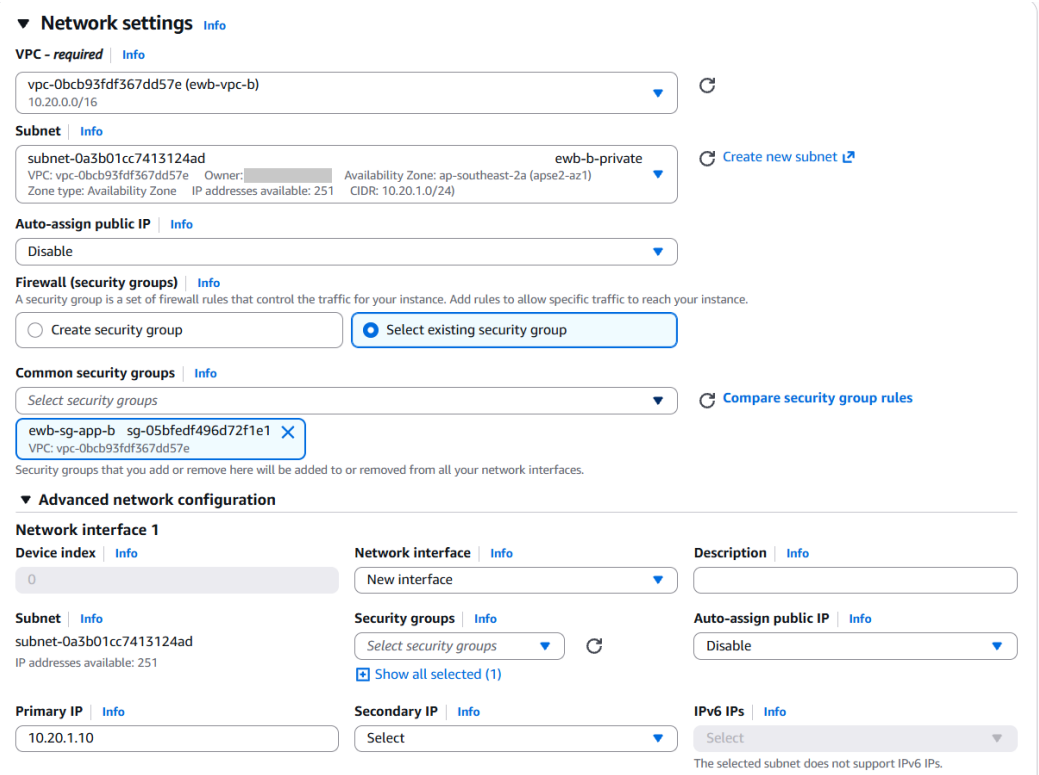


Figure 15. *ewb-app-b launch settings: ewb-vpc-b, ewb-b-private, no public IP, ewb-sg-app-b, primary IP 10.20.1.10.*

Checkpoint 02: the instance list shows that only the bastion has a public IPv4 address. App-A and App-B have none.

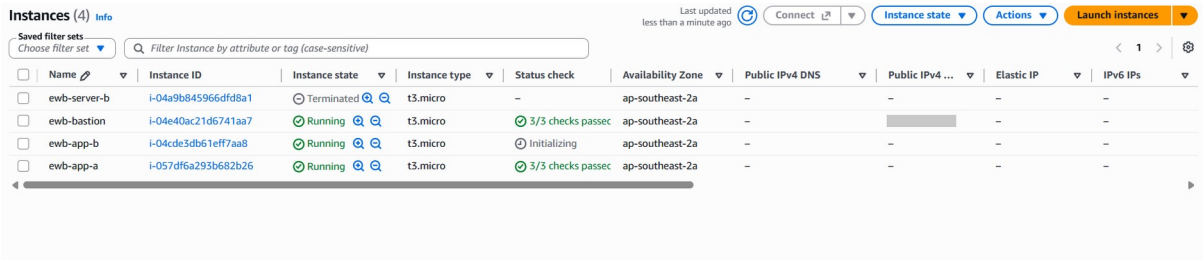


Figure 16. *Instances list (Test 05, isolation): public IPv4 present on ewb-bastion only (blurred); ewb-app-a and ewb-app-b have none.*

SSH configuration on the laptop

SSH was run from Ubuntu on WSL. The key file was moved to ~/.ssh with permission 400, and ~/.ssh/config uses ProxyJump so that the private servers are reached through the bastion without copying the key to any instance.

```
Host ewb-bastion
    HostName BASTION_PUBLIC_IP
    User ec2-user
    IdentityFile ~/.ssh/ewb-key.pem
    IdentitiesOnly yes

Host ewb-app-a
    HostName 10.10.2.10
    User ec2-user
    IdentityFile ~/.ssh/ewb-key.pem
    IdentitiesOnly yes
    ProxyJump ewb-bastion

Host ewb-app-b
    HostName 10.20.1.10
    User ec2-user
    IdentityFile ~/.ssh/ewb-key.pem
    IdentitiesOnly yes
    ProxyJump ewb-bastion
```

```
jinu@Mjolnir:~$ ssh ewb-bastion
The authenticity of host [redacted] can't be established.
ED25519 key fingerprint is: SHA256:rOEzlifquSFHiKoXvpLm5iO8oDsht5H9cnavrmOthk4
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added [redacted] (ED25519) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
   '      #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
[ec2-user@ip-10-10-1-51 ~]$
```

Figure 17. *Laptop to bastion: ssh ewb-bastion reaches the prompt ec2-user@ip-10-10-1-51 (public IP blurred).*

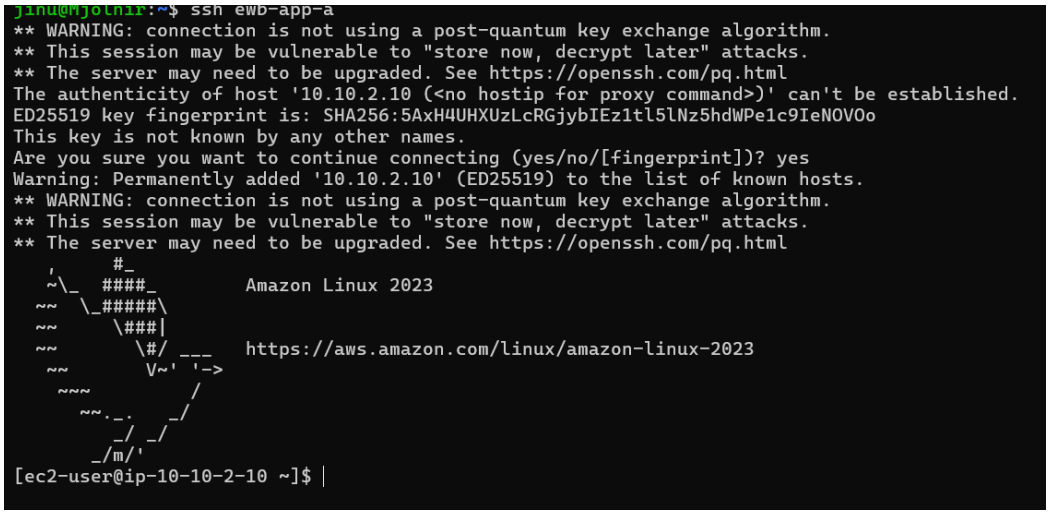


Figure 18. *Laptop to App-A through the bastion: ssh ewb-app-a reaches ec2-user@ip-10-10-2-10 (last login from 10.10.1.51, the bastion).*

6. VPC peering connection

The peering connection was requested from ewb-vpc-a (requester) to ewb-vpc-b (accepter) in the same account and Region. The request must be accepted even when both VPCs belong to the same account. It was created as ewb-peering-a-b and its Name tag was changed to ewb-a-b-peer to match the project naming.

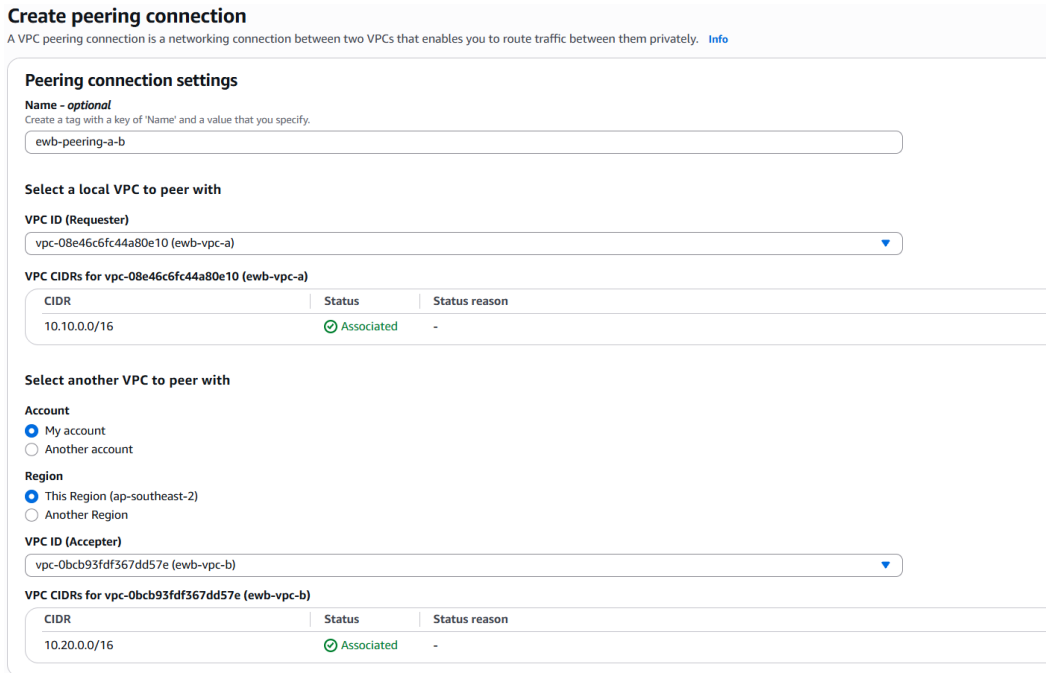


Figure 19. *Create peering connection: requester ewb-vpc-a (10.10.0.0/16), My account, This Region, accepter ewb-vpc-b (10.20.0.0/16).*

Checkpoint 03: after the request was accepted, the status changed to Active. Peering alone does not make ping or SSH succeed, so the routes in section 3.4 were added on both sides.

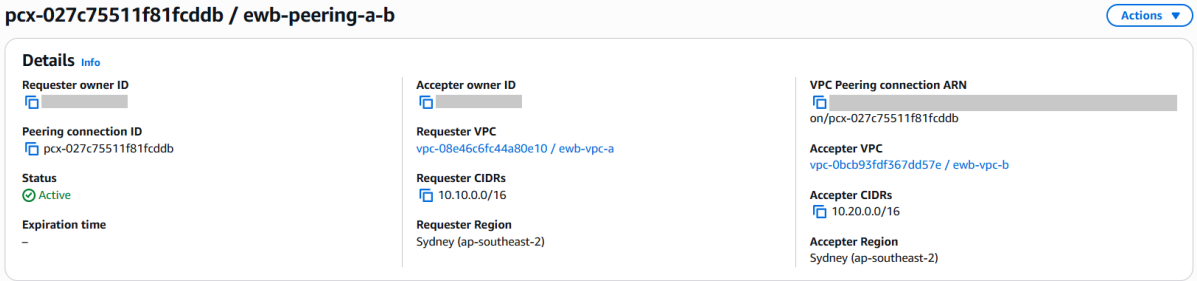


Figure 20. *Peering connection pcx-027c75511f81fcddb: status Active, requester ewb-vpc-a 10.10.0.0/16, accepter ewb-vpc-b 10.20.0.0/16, Sydney.*

7. Verification

Every test below was run from inside the instances using private destination IPs. The screenshots show real terminal output with the host prompt visible.

Test	Where / command	Expected	Actual
01 Baseline	App-A: ping -c 4 10.20.1.10 (before the private peering routes)	No replies	4 sent, 0 received, 100% loss
02 Forward path	App-A: ping -c 4 10.20.1.10 (after routes)	4 replies, 0% loss	4 received, 0% loss
03 Reverse path	App-B: ping -c 4 10.10.2.10 (after routes)	4 replies, 0% loss	4 received, 0% loss
04 Private SSH	Laptop: ssh ewb-app-b	Shell on App-B through the bastion	Connected: ec2-user@ip-10-20-1-10
05 Isolation	EC2 instance list: App-A and App-B	No public IPv4	Public IPv4 empty on both

Test 01: baseline (before the peering routes)

The peering connection existed and was Active, but ewb-rt-a-private and ewb-rt-b-private had only their local routes. The ping from App-A therefore failed. This shows that an Active peering connection alone does not give connectivity: routes are also needed.

```
[ec2-user@ip-10-10-2-10 ~]$ hostname
hostname -I
ping -c 4 10.20.1.10
ip-10-10-2-10.ap-southeast-2.compute.internal
10.10.2.10
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3151ms

[ec2-user@ip-10-10-2-10 ~]$ exit
logout
Connection to 10.10.2.10 closed.
jinu@Mjolnir:~$
```

Figure 21. *ewb-app-a (10.10.2.10) | Test 01 baseline | ping 10.20.1.10: 4 transmitted, 0 received, 100% packet loss.*

Test 02: forward path, App-A to App-B

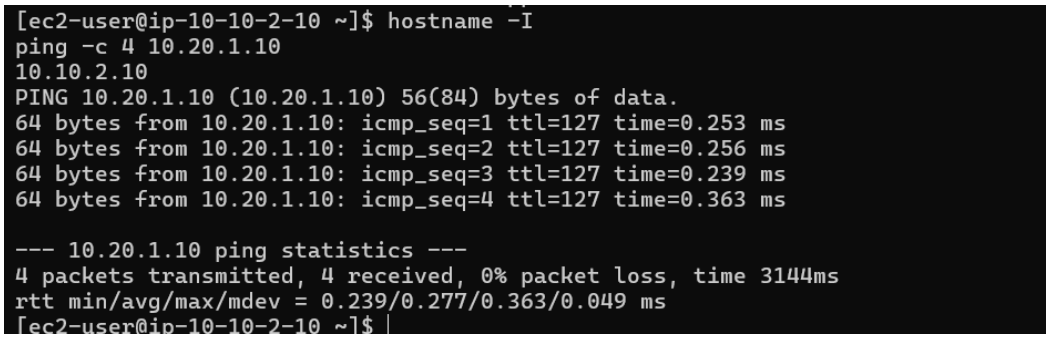


Figure 22. *ewb-app-a (10.10.2.10) | Test 02 forward path | ping 10.20.1.10: 4 transmitted, 4 received, 0% packet loss.*

Tests 03 and 04: reverse path and private SSH to App-B

ssh ewb-app-b reached App-B through the bastion. From App-B, a ping to App-A (10.10.2.10) received all four replies.

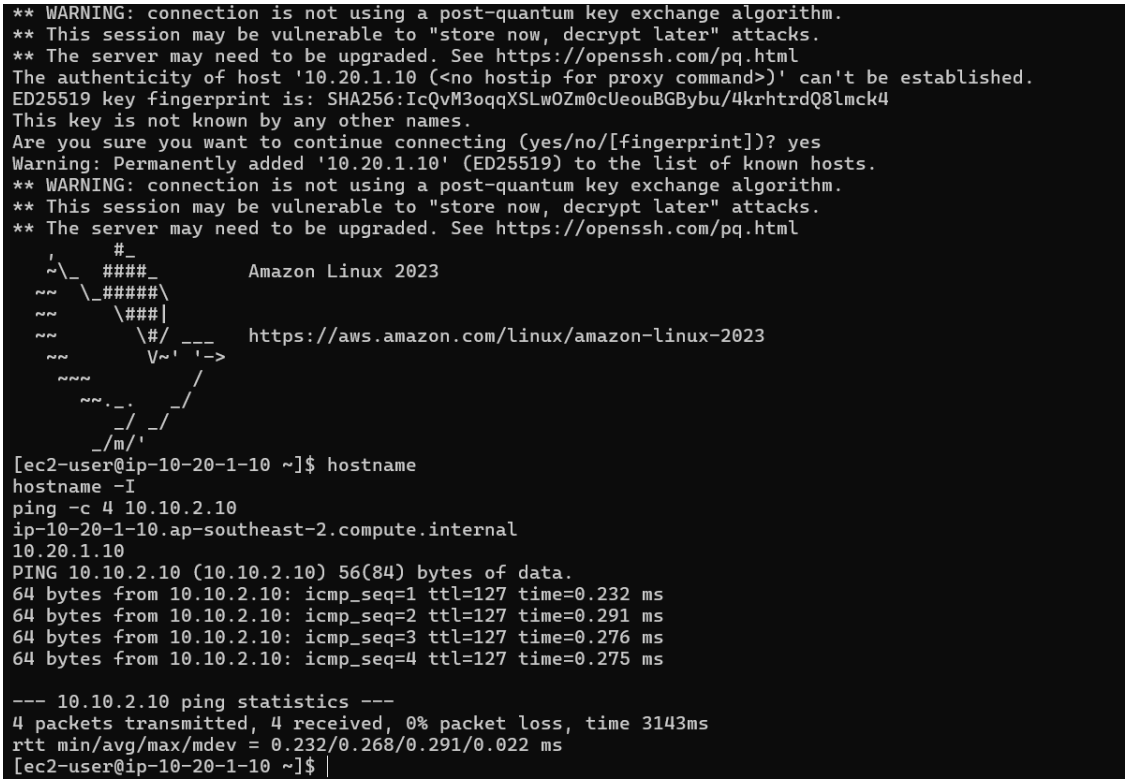


Figure 23. *ewb-app-b (10.20.1.10) | Test 04 SSH through the bastion and Test 03 reverse path | ping 10.10.2.10: 4 received, 0% packet loss.*

8. Controlled failure and recovery

To prove that the security group rule is what allows the ping, only the ICMP Echo Request rule from 10.10.2.10/32 was removed from ewb-sg-app-b. The SSH rule was kept. After a short wait the forward ping from App-A was repeated. The exact rule was then restored and the ping was repeated again.

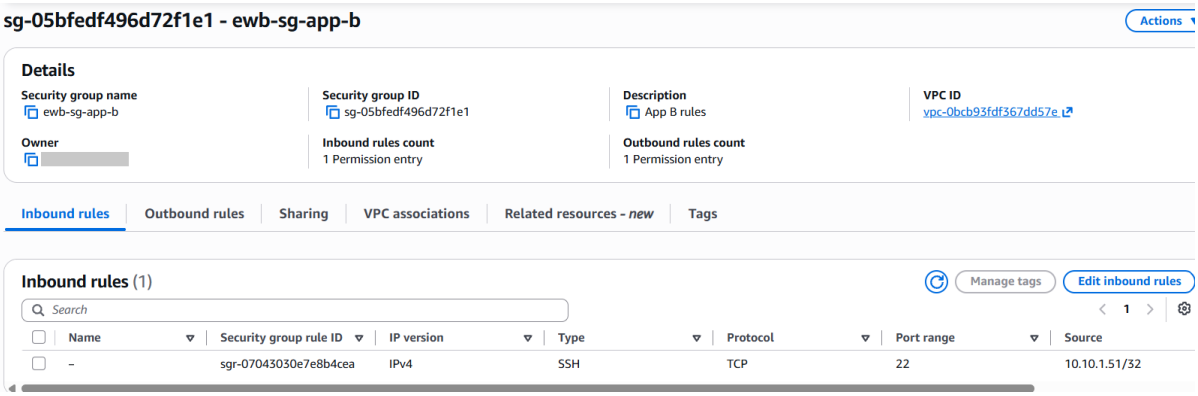


Figure 24. *ewb-sg-app-b | Inbound rules (1): only SSH from 10.10.1.51/32 remains; the ICMP Echo Request rule is removed.*

```
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3091ms

[ec2-user@ip-10-10-2-10 ~]$
```

Figure 25. *ewb-app-a | controlled failure | ping 10.20.1.10: 4 transmitted, 0 received, 100% packet loss.*

The rule was then restored with the same settings: Custom ICMP IPv4, Echo Request, source 10.10.2.10/32. The ping succeeded again.

```
--- 10.20.1.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3117ms
rtt min/avg/max/mdev = 0.241/0.267/0.298/0.021 ms
[ec2-user@ip-10-10-2-10 ~]$
```

Figure 26. *ewb-app-a | recovery after restoring the rule | ping 10.20.1.10: 4 transmitted, 4 received, 0% packet loss.*

> **What this shows**
>
> The peering connection and both routes stayed unchanged, yet traffic stopped when the destination security group no longer allowed Echo Request from App-A, and resumed when the rule came back. Routing decides whether packets can travel; security groups decide whether they are allowed in.

9. Forward path, return path and key concepts

Forward path (App-A to App-B)

1. App-A (10.10.2.10) sends an Echo Request to 10.20.1.10.
2. The route table of ewb-a-private matches 10.20.0.0/16 and sends the packet to the peering connection.
3. The packet crosses the peering connection into ewb-vpc-b and arrives at App-B.
4. ewb-sg-app-b allows the Echo Request because the source 10.10.2.10/32 is in its inbound rules.

Return path (App-B to App-A)

1. App-B sends the Echo Reply to 10.10.2.10.
2. The route table of ewb-b-private matches 10.10.0.0/16 and sends the reply to the peering connection.

3. The reply reaches App-A. Because security groups are stateful, the reply is allowed without a separate inbound rule.

Be ready to explain

1. Why is the return route needed? The request reaches App-B only if App-A's subnet has a route to VPC B, but the reply must find its way back to 10.10.2.10. If ewb-rt-b-private had only its local route, the reply would be dropped and the ping would fail. A one-way route is not enough.

2. Why is the App-B SSH source the bastion private IP? With ProxyJump, the laptop connects to the bastion, and the bastion opens a separate SSH connection to App-B. App-B therefore sees the bastion's private address 10.10.1.51 as the source, not the laptop's public address.

3. Why does an Active peering not guarantee a successful ping? Peering only creates the link. Traffic still needs routes on both sides and security group rules that allow it. The baseline test (Active peering, no private routes) failed, and the controlled failure test (routes in place, ICMP rule removed) failed.

4. Can VPC B use VPC A's internet gateway over peering? No. VPC peering is one-to-one and non-transitive, and it does not forward traffic to an internet gateway in the other VPC. VPC B has no internet gateway and no internet route, so it has no internet access.

10. Cleanup evidence

After the tests, the evidence was saved and all lab resources were removed in dependency order: instances, peering routes and the peering connection, security groups, subnets, route tables, the internet gateway and finally both VPCs. The default VPC was not touched.

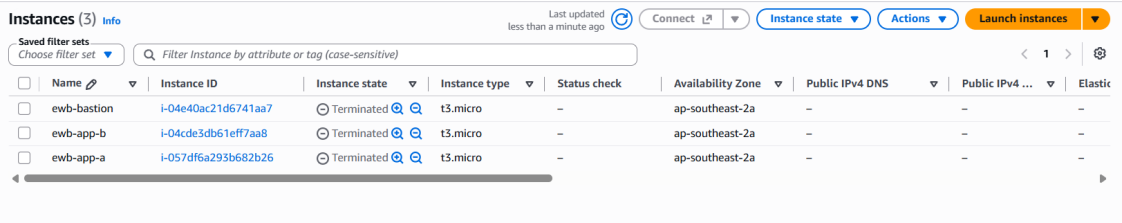


Figure 27. *EC2 instances: ewb-bastion, ewb-app-b and ewb-app-a all in the Terminated state.*

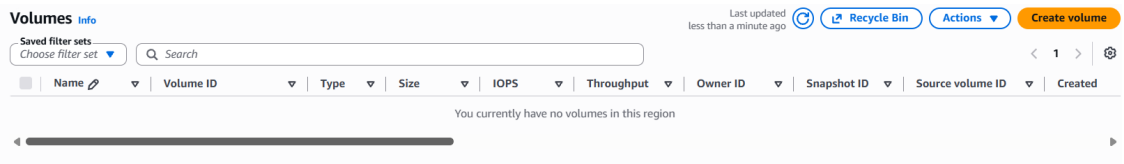


Figure 28. *EBS volumes: no volumes remain in the region.*

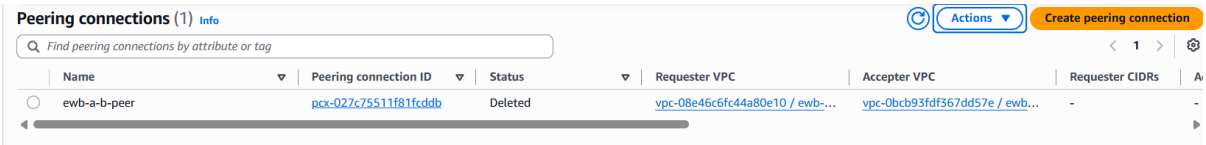


Figure 29. *Peering connection ewb-a-b-peer (pcx-027c75511f81fcddb): status Deleted.*

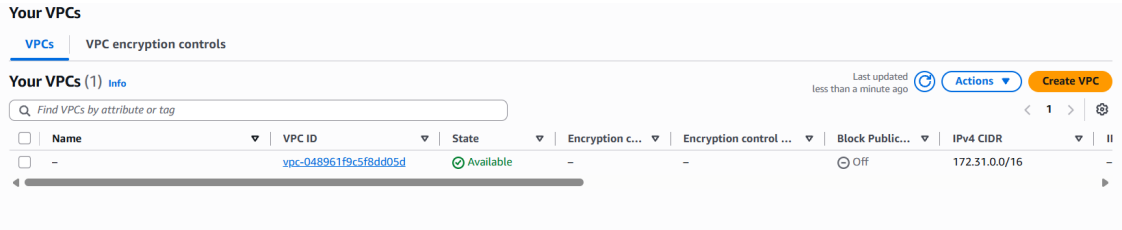


Figure 30. *Your VPCs: only the default VPC (172.31.0.0/16) remains; ewb-vpc-a and ewb-vpc-b are deleted.*

The EC2 key pair was deleted and the local copies of the private key were removed from the laptop. No Elastic IP or NAT gateway was created, so no chargeable resource is left. The cost and usage page showed $0.00 for the current month.

11. What I learned

- Choosing CIDR ranges that do not overlap is a prerequisite for peering. 10.10.0.0/16 and 10.20.0.0/16 do not overlap.
- A public subnet needs an internet gateway route and the instance needs a public address. The private servers need neither.
- Peering needs three things to work: an Active connection, routes on both sides and matching security group rules.
- Testing for failure is as important as testing for success. The baseline and the controlled failure showed which layer decides the result.
- A bastion host with SSH ProxyJump lets me administer private servers without giving them public IPs or copying keys onto them.
- Deleting resources in the right order avoids dependency errors, and cleaning up prevents unexpected charges.